



Mohsen Zeinali (X9465025)

Robotic Process Automation and Its Use Cases

Course:	Robotic Process Automation
Degree:	Master in Robotics
Supervisors:	Tharanath Purushothama
Year:	2026

Robotic Process Automation or RPA, is one of those technologies that sounds more complicated than it really is at first. Based on the course materials I understand RPA as a way of using software robots to imitate the actions that a person normally does on a computer. These actions can include opening applications, clicking buttons, typing into fields, reading data from one place, moving it somewhere else, and following the same rules every time. In the materials RPA is explained as a technology for repetitive and rule-based digital work. The robot does not think like a human in the traditional form of RPA. Instead it follows the steps and rules that have been defined for it.

What makes RPA useful is that many organizations still have a lot of routine work inside their processes. Even when a company has modern systems, there are often gaps between systems, repeated data entry tasks, email- spreadsheet updates, and manual checks. A person may spend part of the workday doing work that is important but not especially meaningful, such as copying values from one system to another or sending the same type of notification again and again. RPA is useful because it can take over that kind of work. It does not replace the whole business process but it can automate the repetitive parts.

From the study materials one of the most important ideas is that RPA is best suited for tasks that are rule-driven, repetitive, voluminous, and digitally driven. I found this list very practical because it helps separate good automation ideas from bad ones. If a task changes all the time then RPA may not be the right solution. But if the steps are stable and happen often, RPA can be a very strong fit. This is an important point because sometimes people talk about automation as if everything should be automated.

I also learned that RPA is valuable to organizations because it can improve speed, accuracy, and service quality. A robot can work much faster than a human when the task is repetitive and clearly defined. It can also work without fatigue which means it does not get distracted. In the materials, RPA is described as a way to improve execution speed and productivity and this makes sense to me because a robot can handle routine steps almost immediately once it is triggered. For example, if a task requires logging into a system, copying fields and sending a message, a robot can perform those steps very quickly.

Accuracy is another major benefit. When people do the same boring task many times mistakes happen naturally. Someone may forget a field or paste data into the wrong place. A robot follows the rules it has been given, so if the process is designed correctly it can reduce those manual errors. This does not mean RPA is magically perfect. One of the most useful warnings from the materials is the “garbage in, garbage out” idea. If the rules or process design are wrong then the robot will perform the wrong action very efficiently. So RPA improves accuracy only when the process itself is well understood and the automation logic is sound.

Another benefit that stood out to me is that RPA can free employees to spend more time on higher value work. The materials say that RPA should not be understood simply as a tool for replacing people. Instead, it should help people by taking over repetitive actions. I think this is one of the strongest arguments for RPA. In many workplaces employees are capable of analysis, problem solving, and process improvement, but a large part of their time is still spent on low-value routine work. If robots can take on those routine steps then people can focus more on tasks that need creativity or personal interaction.

RPA is not only a technical tool but part of a broader business strategy. In the review of the business-oriented module, the role of managers is emphasized. A manager needs to understand the business problem first, then coordinate between business and then validate whether the solution actually solves the original problem. I think this is very realistic. Many automation projects probably fail not because the software is bad, but because the wrong process was selected or the people involved were not aligned.

Another important idea in the materials is the concept of the citizen developer. This was one of the most interesting parts for me because it changes the usual assumption that automation must always be built only by programmers or IT specialists. In the materials, UiPath is described as a low-code tool designed for people who are not traditional programmers. The lecture also explains that citizen developers are people on the business side who understand the process well and can build the core business logic of the automation. I think this idea is powerful because the people who perform the work every day usually know the real pain points better than anyone else. They know which steps are repeated, where delays happen and which tasks are frustratingly manual.

The Wärtsilä case study made this point even stronger. What I learned from that guest lecture is that large-scale RPA success is not only about software features. It is also about governance and empowerment. In the lecture the speaker describes a model where business people are trained to create the first version or minimum viable product of an automation and then a central team supports quality assurance, integration, and the extra elements needed for wider use. I found this model very practical. If only the core RPA team builds everything, they may understand the tool well but not know the business process in enough detail. On the other hand, if business users build everything completely alone, they may know the process very well but create something that is hard to scale or maintain. The best results come when these two groups work together.

The materials also gave a realistic picture of how RPA grows inside a company describing a large organization with many automations, multiple robot machines, and a need to understand how all those automations connect to broader business processes. I found this especially useful because it showed that automation creates its own management challenge. Once an organization has many bots it is not enough to know that each bot works individually. The company also needs visibility into the bigger process picture including where RPA fits and where other technologies such as agents or process mining may help.

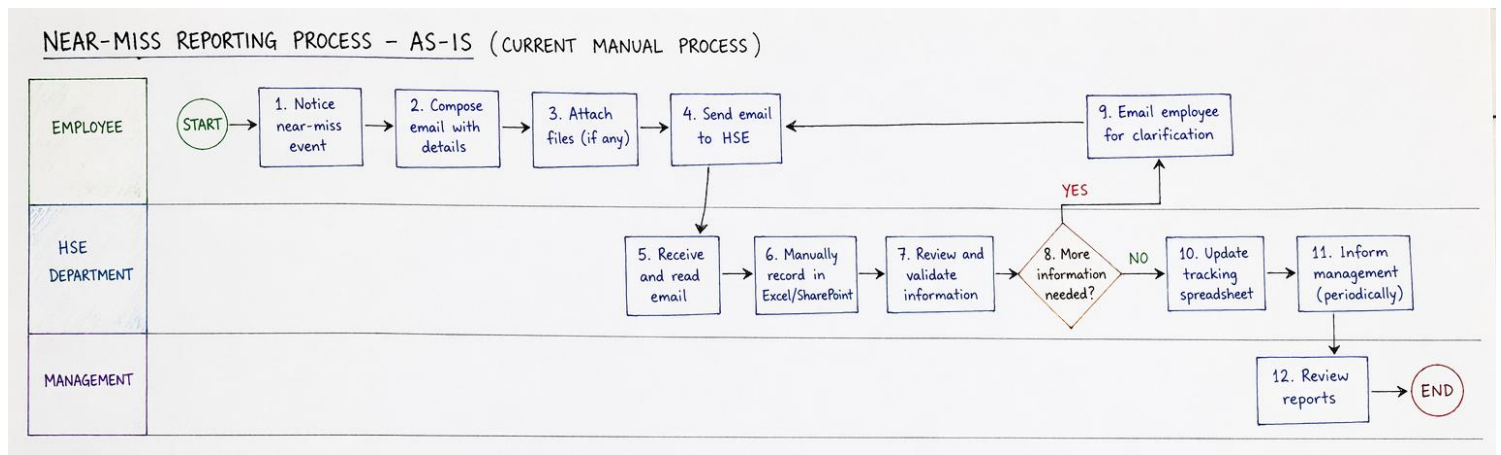
Before building a bot it is necessary to understand why the process should be automated, which kind of process is appropriate and what the expected output should be. In a way, the course setup mirrors what organizations should do: first understand the business value then build the solution carefully.

One of the most memorable parts of the materials was the demonstration of real business processes. The near-miss safety reporting example was especially useful because it showed an automation that has a practical operational purpose. In that example, employees report near-miss cases into one system, and the RPA checks for new entries, fills the data into another system, creates an Excel output and sends an email when new cases are added. I like this example because it is easy to understand but still shows the real value of automation. The robot is not doing something abstract.

For my example business process, I choose the near-miss safety:

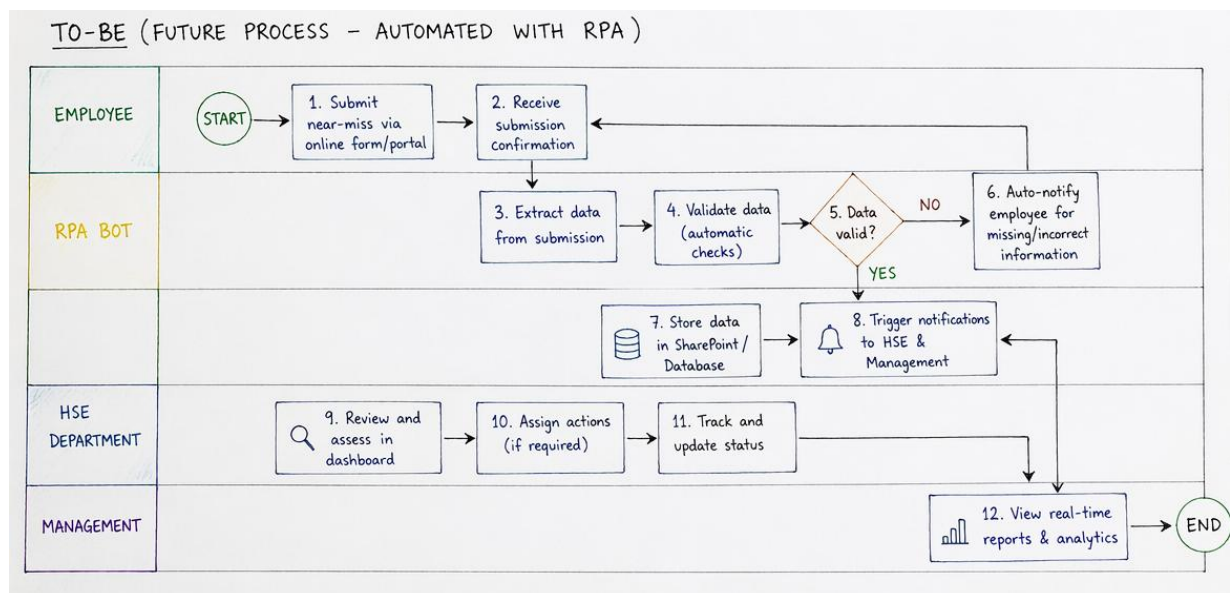
As-Is flowchart (current process):

1. Employee notices and reports a near-miss case in the reporting system
2. New case is stored in the source database
3. Staff member checks new entries
4. Staff member opens the target system
5. Staff member transfers the near-miss data into the target system
6. Staff member prepares an Excel output
7. Staff member sends an email notification to the relevant department.



To-Be flowchart (after automation):

1. Employee notices and reports a near-miss case in the reporting system
2. Robot checks the source database for new entries
3. Robot opens the target system and transfers the required data
4. Robot creates the Excel output automatically
5. Robot sends an email notification to the department.



I would ask: where does the process start and where does it end? In this case, the process starts when an employee submits a near-miss report and ends when the required information has been entered into the target system and the notification has been sent. Defining the boundaries is important because otherwise the automation can grow too wide and become confusing. A good citizen developer should select a process that is manageable and clearly repeatable.

The second step would be to understand the rules, inputs, and outputs. The lecture materials emphasize that citizen developers need to understand the business logic well, and this step is where that matters most. I would identify what counts as a new near-miss case, which fields must be transferred, which system screens are involved, what the Excel output should contain, who should receive the email, and what should happen if required information is missing. I would also clarify the trigger: should the robot run on a schedule, on demand, or whenever a new event happens? In the lecture, triggers such as email, scheduled runs, and event-based actions are all mentioned, so choosing the right trigger is part of good design.

The third step would be to map the process in a simple and visual way before building anything. Even if the final assignment does not require full enterprise development detail, I think a citizen developer still needs a clean sequence of actions. For this process, the map would include checking the database, opening the target system, entering the data, generating the output, and sending the notification. This step matters because it forces the developer to think about the real order of actions rather than jumping too quickly into the tool.

The fourth step would be to review exceptions and stability. Even though the assignment says there is no need to describe full enterprise development or deployment cycles, I think it is still necessary to mention practical checks. For example, what if the target system is unavailable? What if the record is incomplete? What if the same case appears twice? What if the screen layout changes? Since RPA is sensitive to process changes and interface changes, even a citizen developer should think about the most likely error situations. The lecture made it clear that maintenance becomes necessary when systems or processes change, so stability should be considered from the start.

The fifth step would be to make the automation flow clear and usable for the wider team. This connects with what the Wärtsilä lecture says about collaboration between business and the core team. A citizen developer may know the business logic, but for the automation to become more robust and available in the robot environment it may need support from a more central team. That team can help with quality assurance, integration, triggering, and maintenance responsibilities. I think this is one of the best lessons from the materials: citizen development does not mean working alone. It means contributing business knowledge to automation in a structured way.

Looking at this example, I can see why RPA is attractive. The process is repetitive and rule-based. It involves moving data between systems and sending notifications. This is exactly the kind of work that often annoys employees because it must be done carefully but does not require deep human judgment every time. Automating it can improve consistency and make the process faster.

At the same time, the example also reminds me that RPA is part of a larger process not the whole answer to everything. The employee still needs to identify and report the near miss. People still need to decide what actions to take based on that information. The robot helps by removing repetitive transfer work but the meaningful safety decisions remain with humans. Automation should support human work not just replace the parts that require awareness and responsibility.

Overall, The lesson I take from the materials is that successful automation starts with understanding the process not the tool. Tools like UiPath are important because they make automation more accessible, but the real value comes from understanding what the organization is trying to improve, which rules define the work and where humans should remain in control.

In conclusion, Robotic Process Automation is useful because it allows software robots to take over repetitive, rule-based tasks that would consume human time and attention. Organizations benefit through faster execution, better consistency, improved accuracy, and the ability to free employees for more valuable work. RPA is especially well suited to processes with stable rules, digital inputs, and clear outputs. However, it is not a perfect solution for every task. It cannot replace human judgment and it must be designed, supported and maintained carefully. The provided materials show that RPA works best when business knowledge and automation skills come together. For me, that is the most important insight: RPA is not only about robots doing work. It is about people understanding processes well enough to improve them intelligently.